AEGIS 2.1

How to Start and Adapt a New Project

A file-first operating guide for the eight-stage workflow, deterministic verification, explicit human approval, and optional validated RepoWise discovery.

SCHEMA	INTERFACE	DEFAULT CHECKS
3	Stage forms	External

INTAKE > ALIGNMENT > REQUIREMENT > DESIGN > CHECK > IMPLEMENTATION > VALIDATION > DELIVERY

No migration from schema 2. No compatibility reader for AEGIS 1.x.

1. Prerequisites

- Python 3.11 or newer.
- Git 2.30 or newer.
- Node 18 or newer when the selected host adapter requires it.
- Start the agent from the repository root so host configuration and hooks load.

AEGIS is offline-first and its embedded CLI has no third-party runtime dependencies.

2. Validate the Installation

Windows:

```
.\.agentic\tools\run.ps1 config validate
.\.agentic\tools\run.ps1 config show --json
```

POSIX:

```
bash .agentic/tools/run.sh config validate
bash .agentic/tools/run.sh config show --json
```

The configuration and every run graph must use schema 3. There is no schema-2 migration or AEGIS 1.x compatibility reader.

3. Configure the Workspace Interface

The shipped defaults favor small/offline models:

```
[interface]
stage_form_retention = "discard"
check_execution = "external"
```

Retention may be `versioned`, `latest`, or `discard`. The selected value is captured when a stage is prepared. It affects transient forms only; design, requirements/specs, graph state, evidence, journals, validation summaries, and delivery reports remain durable.

Use `external` when the agent or operator will execute check commands and fill the generated evidence form. Use `managed` when AEGIS should execute imported check commands.

4. Configure Optional Deterministic Gates

Five groups are independent and ship disabled:

```
[gates.build]
mode = "disabled"
commands = []

[gates.typecheck]
mode = "disabled"
commands = []

[gates.lint]
mode = "disabled"
commands = []

[gates.test]
mode = "disabled"
commands = []

[gates.security]
mode = "disabled"
commands = []
```

Set a group to `enforce` only with at least one explicit command. Disabled groups may keep commands for later activation. Execution order is build, typecheck, lint, test, security. Security evidence is limited to configured tool output and is not a general assurance claim.

5. Start Work

```
aegis run start feature
aegis run start fix
aegis run start change
aegis run start adoption
```

The command selects the run and returns an intake form. Open it and add the statement, scope hints, and access mode.

Every normal stage follows:

```
aegis stage prepare
aegis stage apply
aegis stage show
aegis stage close
```

Use `stage add` for each additional requirement or check.

6. Understand the Eight Stages

1. Intake - statement, scope hints, access mode.
2. Alignment - shared understanding, scope, assumptions, acceptance, questions, risk, planned paths, decision.
3. Requirement - atomic observable contracts and companion specifications.
4. Design - alternatives, internal architecture, interfaces/data, selected option, sequence, validation, risks.
5. Check - deterministic requirement-linked checks and red evidence.
6. Implementation - approved design changes and green evidence.
7. Validation - deterministic verification first, then human diff, documentation, style, and manual evidence review.

8. Delivery - preflight/report references and final handoff decision.

Host plan mode can help the conversation, but the imported design and `stage close` journal are authoritative.

7. Red Before Green

In external mode:

```
aegis check prepare
aegis check record
```

In managed mode:

```
aegis check run
aegis check run <check-id>...
```

Check closure requires red evidence. Implementation closure requires the prior red evidence and new green evidence. Fixes must reproduce the observed defect, including relevant state transitions and variants.

8. Validate

```
aegis verify run
aegis verify status
```

Verification selects focused/full regression from impact, runs every enabled gate, invariants, traceability, coverage, and working-tree review, then writes a validation summary. Perform the interpretive diff review only after this.

9. Complete Without Publishing

At delivery, complete the generated form and run:

```
aegis stage apply
aegis stage close
```

The run becomes complete. AEGIS performs no push, PR, merge, release, or publication. Those actions require a separate explicit request.

10. WAEGIS Version Boundary

WAEGIS 1.1 supports AEGIS `>=2.0.0,<3.0.0`. It can generate, inspect, configure, optionally execute, and report all five gate groups. Upgrading an AEGIS 1.x installation across the major boundary is unsupported; make a fresh AEGIS 2.x deployment after preserving project-owned artifacts deliberately.

11. Optional RepoWise Discovery

WAEGIS Core defaults RepoWise to disabled. WAEGIS Complete can install a pinned offline RepoWise 0.37.0 sidecar; either edition can link an external compatible 0.37.x command. During `new`, initial indexing defaults to No. During `adopt`, it defaults to Yes. `--yes` never selects RepoWise or starts a long index.

Inspect or synchronize explicitly:

```
waegis repowise status --repo "<project>"
waegis repowise sync --repo "<project>"
```

AEGIS validates the resulting receipt, graph/state schemas, hashes, Git HEAD, and worktree fingerprint during repository discovery. Valid proposals can fill empty discovery fields but never replace human content. Advisory failures warn; only explicitly required RepoWise evidence blocks.

See `REPOWISE_INTEGRATION.md` for policy, provenance, offline, and licensing details.